UNIVERSIDAD PRIVADA FRANZ TAMAYO

UNIFRANZ - Sede La Paz
Carrera: Ingeniería de Sistemas

INFORME TÉCNICO

Sistema de microservicios para la
gestión de productos de un minimarket

Evaluación práctica - Hito 3
Programación III

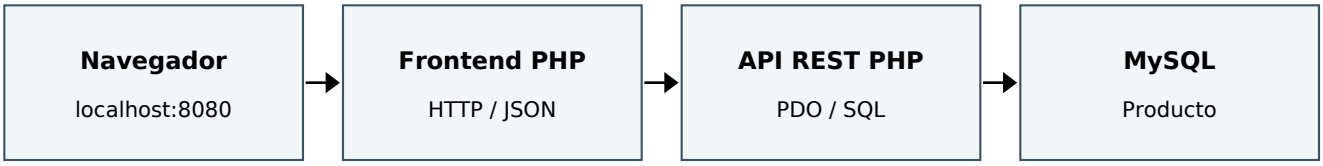
Estudiante:
Álvaro Daniel Sillerico Cruz

La Paz, Bolivia
Octubre de 2026

Alcance de la verificación. Se incluyen evidencias funcionales reales con PHP 8.3.6 y MySQL 8.0.46 en ejecución local. El despliegue previsto utiliza Docker Compose y MySQL 8.4; su arranque y persistencia en contenedores deben comprobarse en la computadora de entrega.

1. Objetivo y arquitectura

Desarrollar una aplicación que permita consultar, registrar, editar y eliminar lógicamente productos de un minimarket. La solución separa la presentación, la API REST y la persistencia, respetando los nombres de base de datos, tabla y columnas exigidos por la evaluación.



Flujo de solicitudes: frontend → API REST → MySQL

1.1. Responsabilidades

Frontend: presenta la tabla, los formularios y la confirmación de eliminación. Utiliza un cliente HTTP para consumir la API. No contiene SQL, no usa PDO y no recibe credenciales MySQL.

Backend: interpreta las rutas y métodos HTTP, valida los campos, procesa las operaciones y responde en JSON. Solo este servicio accede a la base de datos mediante PDO y consultas preparadas.

MySQL: almacena los productos y aplica las restricciones del esquema. Docker Compose configura su volumen persistente y espera la disponibilidad de la tabla antes de iniciar la API.

1.2. Estructura principal

Archivo o carpeta	Responsabilidad
backend/public/index.php	Rutas, respuestas JSON y manejo de errores.
backend/src/Database.php	Conexión PDO con variables de entorno.
backend/src/ProductValidator.php	Campos, fechas, importes y cantidades válidas.
backend/src/ProductRepository.php	Consultas SQL y eliminación lógica.
frontend/public/	Interfaz PHP y estilos CSS.
frontend/src/ApiClient.php	Peticiones HTTP al backend.
database/	Esquema, datos de ejemplo y consultas de comprobación.
docker-compose.yml / tests / docs	Servicios, pruebas, instrucciones y evidencias.

2. Modelo de datos y API REST

La base de datos se denomina **db_minimarket_pro413**. La tabla se denomina **Producto**, conservando la P mayúscula. Se usa InnoDB y codificación utf8mb4.

Campo	Tipo	Regla
id	INT	Clave primaria y AUTO_INCREMENT.
nombre	VARCHAR(100)	NOT NULL.
proveedor	VARCHAR(100)	NOT NULL.
fecha_vencimiento	DATE	NOT NULL.
precio	DECIMAL(10,2)	NOT NULL y CHECK precio >= 0.
stock	INT	NOT NULL y CHECK stock >= 0.
eliminado	TINYINT(1)	NOT NULL, DEFAULT 0 y valores 0 o 1.

2.1. Eliminación lógica

La eliminación conserva físicamente el registro. Las consultas habituales filtran por eliminado = 0. Un producto eliminado deja de estar disponible en el listado, la consulta individual y la edición.

```
UPDATE Producto SET eliminado = 1
WHERE id = :id AND eliminado = 0;

SELECT * FROM Producto WHERE eliminado = 0;
```

2.2. Endpoints

Método	Ruta	Respuesta esperada
GET	/api/productos	200; lista de productos activos.
GET	/api/productos/{id}	200; producto activo.
POST	/api/productos	201; producto creado.
PUT	/api/productos/{id}	200; producto actualizado.
DELETE	/api/productos/{id}	200; cambio a eliminado = 1.
GET	/health	200 si la API accede a la tabla.

Los cuerpos POST y PUT usan JSON con nombre, proveedor, fecha_vencimiento, precio y stock. Se verifican textos de 1 a 100 caracteres, fechas reales, precios de hasta dos decimales y stock entero no negativo. Las respuestas distinguen JSON inválido (400), producto inexistente (404), método no admitido (405), tipo de contenido incorrecto (415), validación (422) y errores de persistencia (503).

3. Configuración y Docker Compose

El archivo docker-compose.yml define los servicios mysql, backend y frontend. Las imágenes de destino son mysql:8.4 y php:8.3-apache. El Dockerfile del backend instala la extensión pdo_mysql. El frontend usa HTTP streams de PHP, sin bibliotecas externas.

Componente	Acceso desde Windows	Acceso entre servicios
Frontend	http://localhost:8080	Contenedor frontend, puerto 80.
API REST	http://localhost:8081	http://backend/api/productos
MySQL	127.0.0.1:3307	mysql:3306

3.1. Variables de entorno

Variable	Configuración
MYSQL_ROOT_PASSWORD	Clave administrativa local; solo la utiliza MySQL.
MYSQL_USER / MYSQL_PASSWORD	Usuario y clave de aplicación. El backend recibe sus valores como DB_USER y DB_PASSWORD.
DB_HOST / DB_PORT / DB_NAME	mysql / 3306 / db_minimarket_pro413.
API_BASE_URL	http://backend/api/productos en el frontend.
FRONTEND_PORT / BACKEND_PORT / MYSQL_PORT	8080 / 8081 / 3307, configurables en .env.

El archivo .env contiene la configuración local y está excluido de Git. Se entrega .env.example con valores de ejemplo y un script que genera claves aleatorias. Las contraseñas no se incorporan al código. Los puertos se publican únicamente en 127.0.0.1 para la práctica local.

3.2. Persistencia e inicialización

El volumen mysql_data se monta en /var/lib/mysql. Los scripts 01-schema.sql y 02-seed.sql se montan como archivos de solo lectura en /docker-entrypoint-initdb.d/. En un volumen nuevo crean la tabla y cinco productos de ejemplo. Estos scripts no se vuelven a ejecutar automáticamente sobre una base ya inicializada.

Los controles de salud verifican MySQL y la API. Con depends_on y service_healthy, el backend espera a MySQL y el frontend espera al backend. docker compose down conserva el volumen; agregar -v lo elimina y ocasiona pérdida de los datos del volumen.

3.3. Comunicación y protección básica

El navegador recibe HTML del frontend. El frontend realiza las solicitudes JSON en el servidor, por lo que no requiere CORS. Los formularios incluyen token CSRF, los textos se escapan al mostrarse y el backend usa consultas preparadas. Esta práctica no incorpora autenticación ni roles porque no forman parte de la consigna.

4. Preparación, ejecución y repositorio

4.1. Requisitos y comprobación

En Windows se requieren Docker Desktop funcionando, Git y un navegador. Para el modo Docker no se requiere instalar PHP, Java, Node, XAMPP ni MySQL local. Si se utiliza WSL 2, se debe comprobar su instalación y la virtualización según los requisitos oficiales de Docker Desktop.

```
git --version
docker --version
docker compose version
wsl --version
docker info
```

4.2. Arranque desde la carpeta del proyecto

```
powershell -NoProfile -ExecutionPolicy Bypass -File .\scripts\preparar.ps1
docker compose up -d --build
docker compose ps
```

Abrir http://localhost:8080. La API responde en http://localhost:8081/api/productos y su comprobación de conexión está en /health. La construcción de las imágenes prepara PHP; no hay una compilación adicional. El README también documenta PHP local con MySQL en Docker.

4.3. Comprobaciones y detención

```
docker compose exec backend php tests/validation.php
docker compose exec backend php tests/api-smoke.php
docker compose logs --tail=80
docker compose down
```

La prueba API deja un producto eliminado lógicamente para consultar su ID en MySQL. Para comprobar persistencia, iniciar nuevamente con docker compose up -d y verificar que los cambios se mantienen. Esa comprobación en contenedores queda pendiente en la computadora de entrega.

4.4. Publicación del código

Crear un repositorio vacío en GitHub y sustituir TU_USUARIO por la cuenta real. Antes del commit, revisar que .env no aparezca entre los archivos preparados. No se creó ni publicó un repositorio remoto durante esta preparación.

```
git init
git branch -M main
git add .
git status
git commit -m "Implementa minimarket con microservicios"
git remote add origin https://github.com/TU_USUARIO/minimarket-hito3.git
git push -u origin main
```

Entregar el enlace web accesible del repositorio y este informe. El repositorio debe incluir backend, frontend, database, docker-compose.yml, .env.example, README.md, tests y docs.

5. Evidencias de ejecución: listado

Las siguientes capturas se obtuvieron al ejecutar el proyecto con PHP 8.3.6 y MySQL 8.0.46 como procesos locales de prueba. El frontend consumió la API por HTTP; la API consultó MySQL real. Estas capturas acreditan el funcionamiento del código y no representan una ejecución de Docker Desktop.

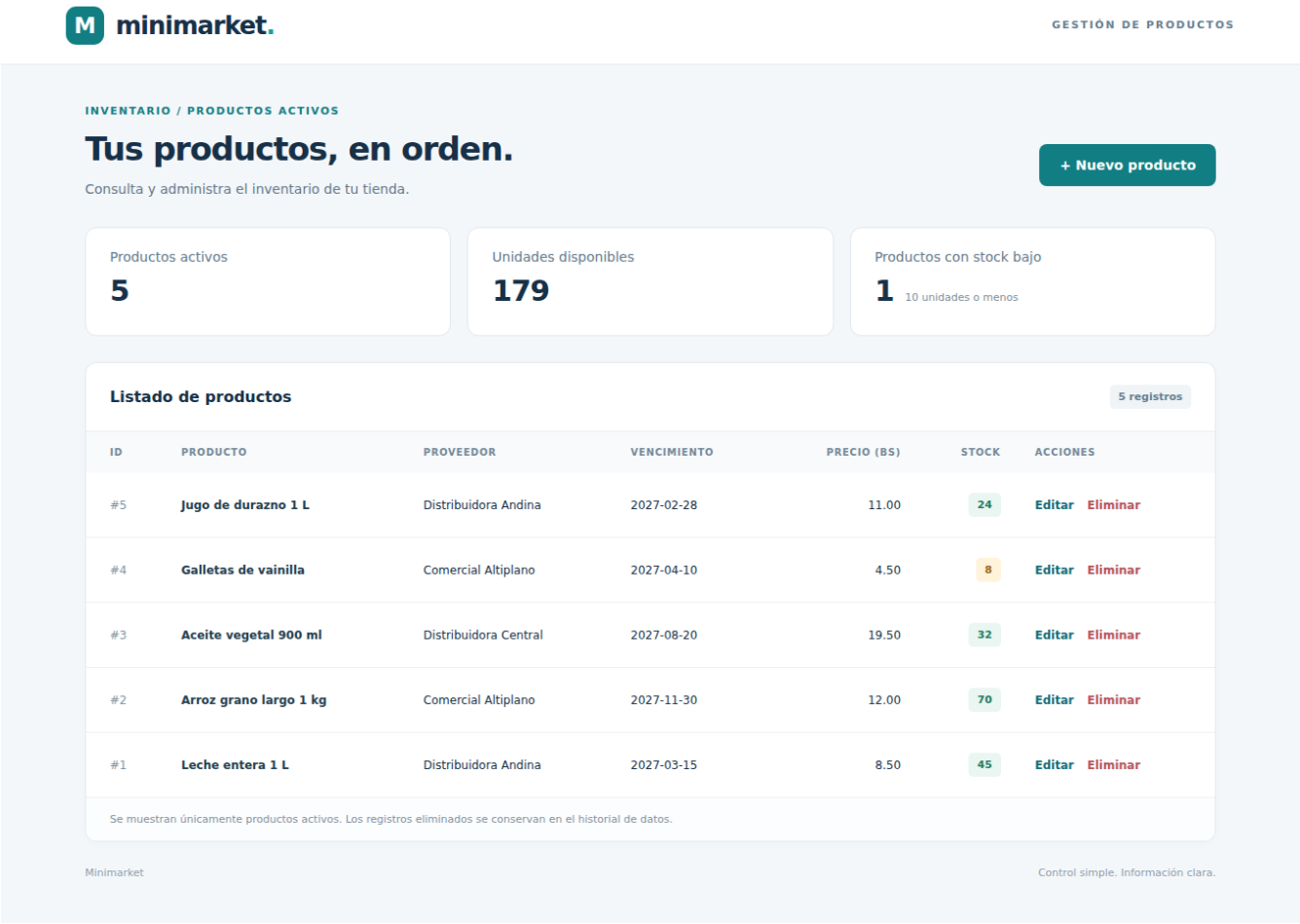


Figura 1. Listado inicial con cinco productos activos. Se muestran ID, nombre, proveedor, fecha de vencimiento, precio, stock y las acciones Editar y Eliminar.

El frontend obtiene la información mediante GET /api/productos. Los registros de ejemplo provienen de 02-seed.sql. El total de unidades inicial es 179 y la interfaz identifica un producto con 10 unidades o menos. No hay acceso directo del frontend a la base de datos.

6. Evidencias de ejecución: creación

Se registró el producto Café prueba Hito 3, con proveedor Distribuidora Central, vencimiento 2027-12-31, precio de Bs 25,50 y stock de 20 unidades. La operación se realizó desde el formulario de la interfaz y produjo una respuesta 201 en la API.

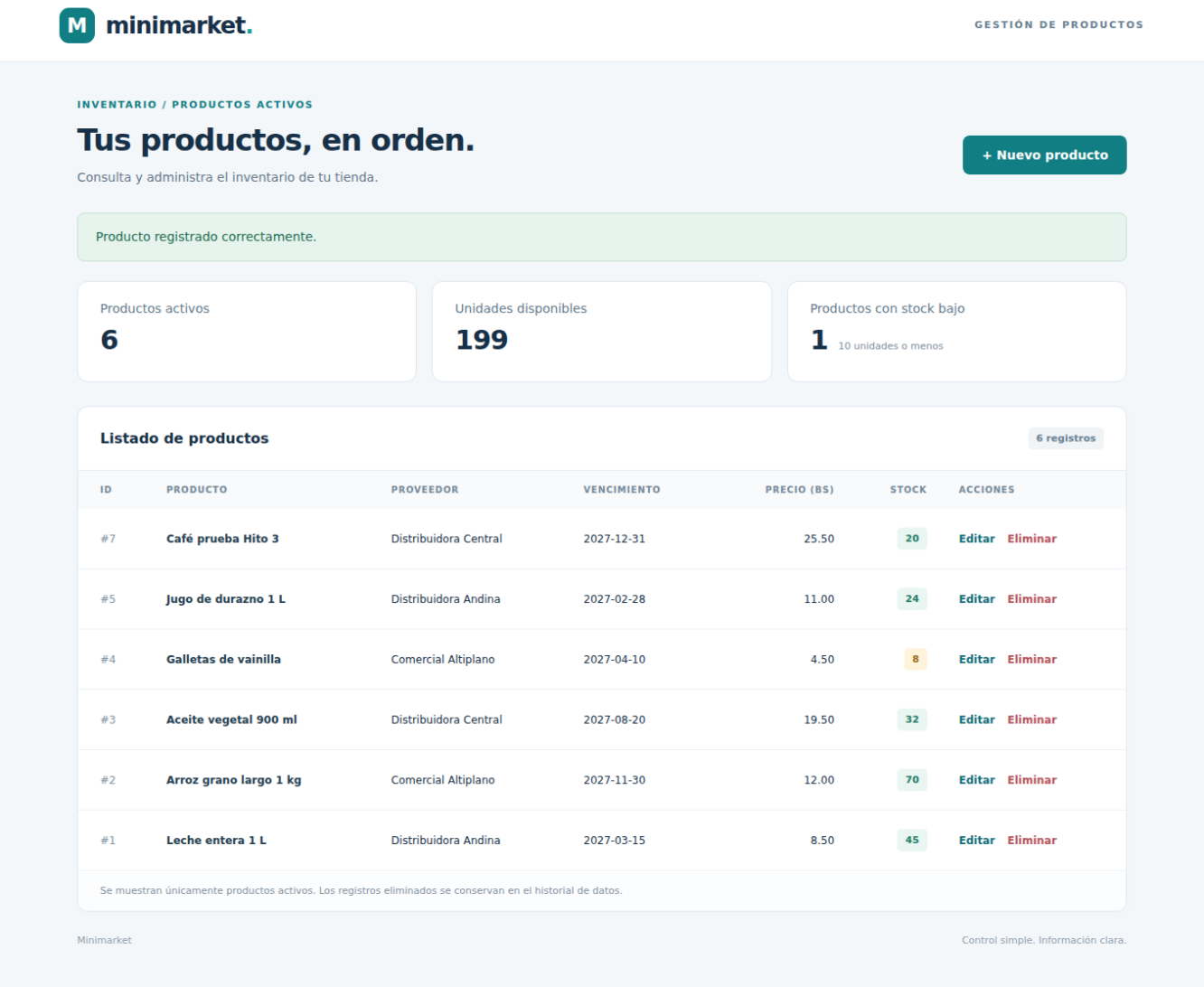


Figura 2. Mensaje de registro correcto y nuevo producto con ID 7 en el listado activo. El formulario previo al guardado se conserva como 02-formulario-creacion.png en docs/evidencias.

El ID 6 pertenece a la prueba automática de la API y ya estaba eliminado lógicamente. Por eso el producto creado desde la interfaz recibió el ID 7. En otra instalación los IDs pueden diferir según las operaciones realizadas.

7. Evidencias de ejecución: edición

Se seleccionó Editar para el mismo producto y se modificaron el precio a Bs 27,00 y el stock a 35 unidades. La interfaz envió PUT /api/productos/7. La API validó los datos y ejecutó la actualización sobre el registro activo.

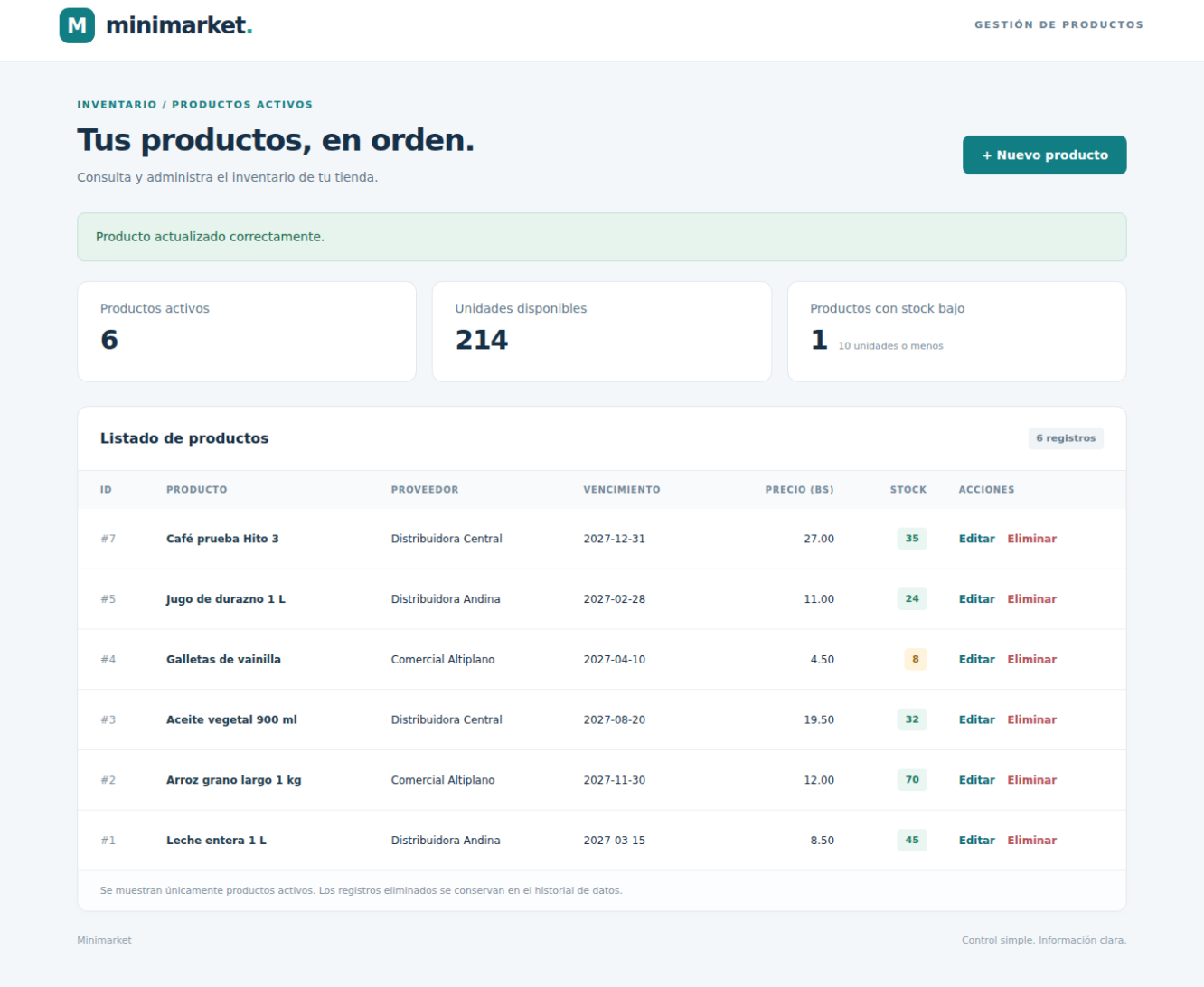


Figura 3. Mensaje de actualización correcta. El producto conserva su ID y muestra los nuevos valores de precio y stock.

La prueba automatizada también comprobó que guardar un producto sin cambios devuelve éxito, evitando confundir una actualización sin diferencias con un registro inexistente. No se modifican el ID ni el estado eliminado desde los datos del formulario.

8. Evidencias de ejecución: eliminación lógica

Se confirmó la eliminación del producto ID 7. Desapareció del listado activo y de la respuesta de la API. Una consulta posterior a MySQL comprobó que conserva el precio, el stock y eliminado = 1.

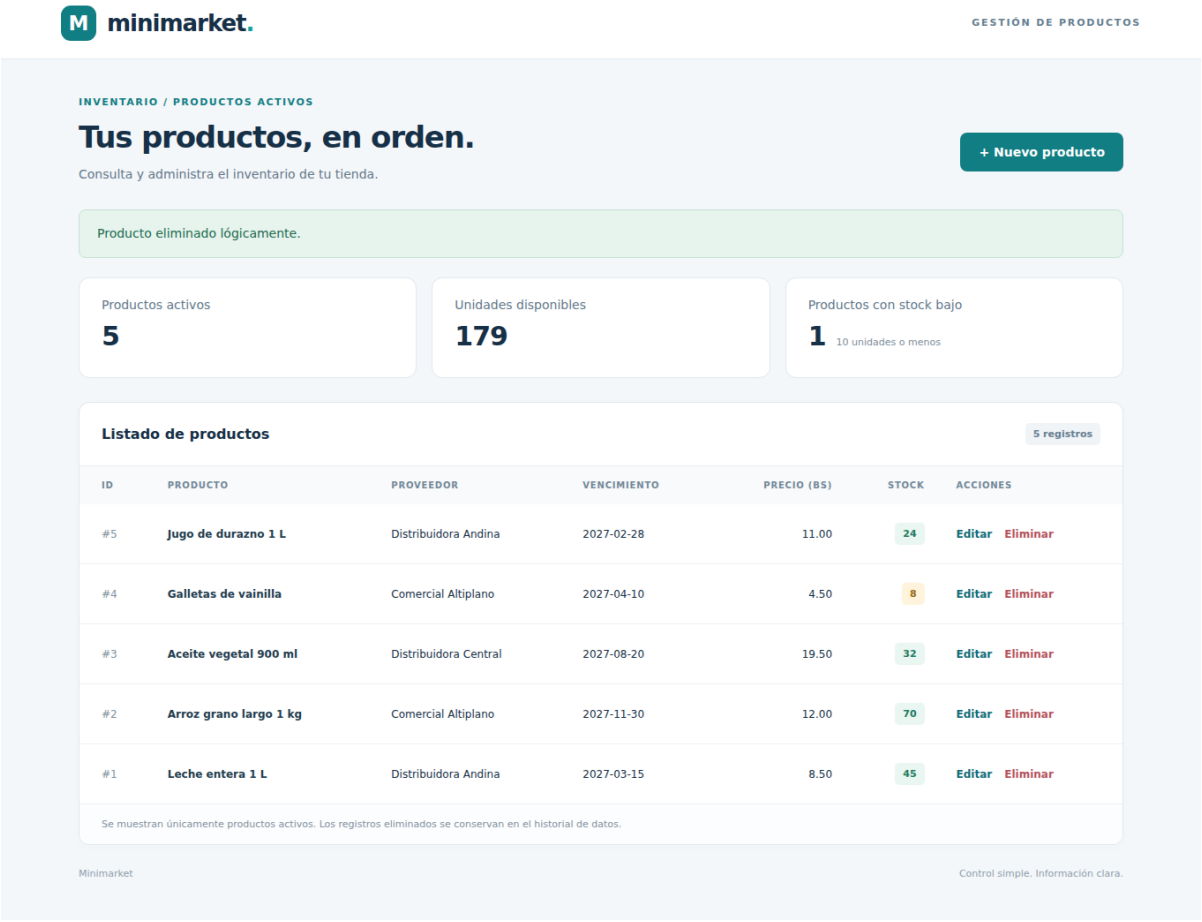


Figura 4. Listado posterior a la eliminación: el producto Café prueba Hito 3 ya no aparece.



Figura 5. Salida real de MySQL exportada a imagen legible. La fila ID 7 continúa almacenada. La salida SQL completa también está en consulta-mysql.txt.

9. Resultados y comprobaciones pendientes

Verificación	Resultado
Sintaxis PHP	9 archivos revisados sin errores.
Validación de datos	13 pruebas correctas.
API REST	13 comprobaciones correctas.
Interfaz web	7 comprobaciones correctas, incluyendo CSRF y vista móvil.
Registro eliminado en MySQL	ID 7 conservado con eliminado = 1.
Configuración Compose	YAML leído correctamente y servicios revisados.
Ejecución en Docker / MySQL 8.4	Pendiente de comprobar en la computadora de entrega.

El entorno de preparación no dispone de motor Docker. No se ejecutó la construcción de imágenes Apache/PHP ni se comprobó el volumen mediante down/up. Las pruebas locales con MySQL 8.0.46 no reemplazan la verificación del despliegue con MySQL 8.4. Los scripts de PowerShell se entregan para Windows y tampoco se ejecutaron en este entorno Linux.

Antes de entregar, ejecutar el README, repetir las cuatro operaciones, verificar la conservación del registro eliminado y adjuntar una captura de docker compose ps con los tres servicios saludables. La guía docs/GUIA_EVIDENCIAS.md contiene la secuencia exacta. El enlace del repositorio debe añadirse después de subir el código a la cuenta del estudiante.

10. Conclusión

Se implementaron y comprobaron en ejecución local las operaciones de consulta, registro, edición y eliminación lógica, con comunicación HTTP entre frontend y API y almacenamiento en MySQL. La solución incluye la configuración Docker y la documentación de puesta en marcha. La validación final de los contenedores y la publicación del repositorio corresponden a la etapa de entrega.

11. Referencias

Consigna de evaluación práctica: PRO-314_Hito3_continua. Documento proporcionado para esta actividad.

Docker. Instalación de Docker Desktop en Windows. https://docs.docker.com/desktop/setup/install/windows-install/

Docker. Control del orden de inicio en Compose. https://docs.docker.com/compose/how-tos/startup-order/

Docker Official Images. PHP. https://hub.docker.com/_/php

Docker Official Images. MySQL. https://hub.docker.com/_/mysql

PHP. Controlador PDO MySQL. https://www.php.net/manual/en/ref.pdo-mysql.php